

OpenKernelForge: Repeatability-Aware Evaluation for LLM-Generated Triton Kernels

Anonymous Authors

Abstract

Generated GPU kernels require evidence of correctness and stable speed, not plausible source alone. We present OpenKernelForge, an artifact-preserving evaluation harness for LLM-generated Triton kernels that combines static policy checks, correctness verification, CUDA-event timing, cache-state perturbation, repeated measurement sessions, session-level uncertainty summaries, and eager and compiler baselines. On a controlled internal fused8 suite, Gemini and OpenAI mini outperform `torch.compile max-autotune` at median, by $1.863\times$ and $1.835\times$, while remaining below PyTorch eager at median; deterministic templates provide the strongest overall floor. A deterministic `bias_relu` candidate also changes from a $1.029\times$ single-run win to a $0.976\times$ repeat median, showing that measurement instability is not specific to model sampling. We additionally preserve a capped KernelBench L1 pilot and one repair pass. A post-hoc adapter audit found that those historical runs replaced the official **ModelNew** lifecycle with a free-function contract and reconstructed reference modules inside timed calls. We therefore retain their artifacts as a failure-characterization case study but exclude their correctness rates and speedups from supported performance claims pending corrected revalidation. The contribution is methodological: generated-kernel evaluation should distinguish correctness, compiler speedup, eager speedup, and repeat-stable speedup, while preserving enough evidence to audit the evaluator itself. We make neither a state of the art claim nor a benchmark-wide KernelBench claim.

1 Introduction

LLMs and coding agents increasingly generate low-level GPU kernels from high-level tensor programs. This capability is useful only if the generated kernels are correct, inspectable, and faster under a measurement protocol that survives reruns. Correctness alone is not enough: a candidate may pass tests while remaining slower than PyTorch eager or a compiler baseline. A single timing run is also not enough. GPU timing is affected by warmup, lazy compilation, synchronization, cache state, clock behavior, and sample variance.

The problem is sharper for generated-kernel systems because they search over candidates. When the selection signal is noisy, a search procedure can select measurement artifacts rather than durable runtime improvements. A result table that reports only the fastest observed run can therefore overstate progress, especially when many candidates were attempted.

OpenKernelForge is built around a narrower claim: generated-kernel systems need an artifact-preserving, repeatability-aware evaluation layer before their speedups are promoted into reports or datasets. The project is not primarily a kernel generator. It contributes an evaluation protocol and artifact discipline: each candidate keeps its prompt or template provenance, raw model output, extracted source, static policy result, verifier result, timing samples, repeatability label, and report row. The harness compares candidates against PyTorch eager, `torch.compile max-autotune`, and deterministic Triton templates that serve as an explicit performance floor.

This paper reports a controlled internal fused8 study and an audit of a capped KernelBench L1 pilot. The fused8 study compares deterministic templates, Gemini, and OpenAI mini under one CUDA-event timing protocol. The KernelBench artifacts cover one Gemini candidate per task and one repair pass, but a post-hoc implementation audit found task-contract and reference-lifecycle defects. We retain that pilot as an evaluator-audit case study rather than supported external performance evidence.

The two components answer different questions. The fused8 study asks how candidate sources compare when tasks and timing are controlled. The KernelBench audit asks whether the external adapter itself satisfies the official state and timing contract. This distinction is central: an evaluation layer must make generated candidates auditable, but it must also expose defects in its own baselines and task adaptation.

Contributions.

- A repeatability-aware measurement protocol for generated Triton kernels using CUDA events, cache-state perturbation, repeated sessions, session-level summaries, materialized compile accounting, and `torch.compile max-autotune`.
- An artifact-preserving verifier pipeline that records prompts, responses, source, policy checks, correctness traces, timing summaries, repeatability labels, and dataset metadata.
- A controlled fused8 study showing why deterministic templates and repeatability labels are necessary baselines for generated-kernel claims.
- A corrected KernelBench adapter contract that initializes persistent reference `Model` and candidate `ModelNew` modules outside timed regions and rejects state-incompatible candidates.
- An artifact-backed audit showing why the historical KernelBench pilot must be revalidated rather than silently accepted or rewritten.

2 Methodology

Task suites. The controlled study uses an internal fused8 suite with eight fused operations: `bias_relu`, `sigmoid`, `add_relu`, `residual`, `bias_gelu`, `row_sum`, `layernorm`, and `rmsnorm`. Each task has a PyTorch reference, deterministic input generation, tolerances, benchmark shapes, and prompt metadata. The external pilot uses official KernelBench L1 tasks [9] at commit 423217d9fda91e0c2d67e4a43bf62f96f6d104f1.

KernelBench subset selection. The historical KernelBench pilot loads a 100-task L1 pool, orders tasks by deterministic family round-robin, and scans that order until 20 feasible tasks are found. Inputs are materialized for each scanned task before timing, and tasks above a configured lower-bound memory estimate are skipped with reason `ESTIMATED_MEMORY_TOO_LARGE`. The recorded 31 skips are therefore skips encountered in the scanned prefix before the selection cap, not a count over every task in the loaded pool. The corrected preflight requests metadata-only tensors first; unsupported metadata execution is skipped unless CPU fallback is explicitly enabled. Its lower-bound estimate accounts for the input-tree copies held by verification, reference and candidate module state, and configured cache-perturbation storage, but not outputs, compiler workspace, all temporaries, or allocator fragmentation. Task definitions are not modified. The selected subset covers convolution 7, matmul 7, pooling 3, and loss 3; it is neither random nor an unbiased L1 sample.

Candidate sources. The fused8 study compares deterministic Triton templates, Gemini 3.1 Flash-Lite, and OpenAI `gpt-5.4-mini`. The affected historical KernelBench run used Gemini only, with one candidate per selected task and one repair pass over eight selected failures. These budgets describe artifact provenance; the audit prevents interpreting their outcomes as model accuracy or repair effectiveness.

Model identifiers are reported as configured API strings. The fused8 Gemini and KernelBench Gemini configs use `gemini-3.1-flash-lite`; the OpenAI mini fused8 config uses `gpt-5.4-mini`. The local artifact package does not preserve provider response `model` fields for the fused8 runs. KernelBench JSONL records preserve the configured `model` field for each candidate but not a separate provider-returned `model-version` field.

Environment. The final KernelBench artifacts record Python 3.12.3 on Linux 6.8, Torch 2.8.0+cu128, Triton 3.4.0, one NVIDIA GeForce RTX 5090, CUDA available, and a passing tiny Triton kernel. The final fused8 campaign was run on the same reported RunPod GPU stack, but the local paper package does not include the fused8 `environment_probe.json` files. GPU and memory clocks, power limits, persistence mode, ECC status, and thermals were not recorded as locked or controlled.

Candidate contract, verification, and timing. Fused8 candidates expose `forward(*args)`. Every official KernelBench task that defines `Model` uses the corrected `ModelNew` contract, including tasks with an empty `state.dict()`. The adapter materializes `get_init_inputs()` once under a fixed seed, clones that snapshot into persistent reference and candidate modules, and passes `get_inputs()` to their forward methods. Both modules move to the target device before verification and timing. Free functions are reserved for local synthetic tasks with an explicit `reference_fn` interface. Verification compares output structure, shape, dtype, finite values, numerical tolerance, and input side effects; a candidate may mutate an input only when the reference produces the same final input state. Static policy checks reject missing entry points, reference calls, high-level Torch fallbacks, unsafe imports/calls, and import-only Triton claims. They are guardrails, not an operating-system sandbox.

Correct candidates are benchmarked using CUDA events [8], 30 warmup iterations, three same-process measurement sessions, cache-state perturbation with a 128 MB CUDA buffer, session-level summaries, and `torch.compile max-autotune` [1]. Fused8 uses 120 measured samples per session. Historical KernelBench baseline validation used 100 samples per session, while candidate and repair runs used 120. Current runs rotate eager, candidate, and compile order between sessions. PyTorch compilation is materialized before runtime sampling; the recorded compile field includes wrapper creation and the first synchronized call.

Historical KernelBench adapter audit. The preserved KernelBench runs predate the corrected `ModelNew` and persistent-reference implementation. Their adapter required free `forward(*args)` candidates and reconstructed `Model` inside reference calls. This violated the official candidate lifecycle for every selected task; parameterized tasks additionally had no route to the reference state. We retain the original records without relabeling them, but treat their correctness and speedup rows as provisional diagnostics pending corrected-adapter revalidation. The internal fused8 study does not use this adapter.

Table 1: Static policy checks. Policy rejects common fallback and escape patterns before execution; it is a guardrail rather than an operating-system sandbox.

Check	Detection method	Purpose
Syntax	Parse with Python AST	Reject unimportable candidates
Forward entry point	Module <code>forward</code> or <code>ModelNew.forward</code> ; reason <code>missing_forward</code>	Enforce task contract
Reference imports	Import names/modules containing task or reference paths; reason <code>imports_reference_or_task_module</code>	Block direct task-reference access
Import allowlist	Reject unsafe modules, noncanonical aliases (<code>disallowed_import_alias</code>), and direct Torch compute imports (<code>disallowed_from_torch_import</code>)	Limit common escape and alias paths
Import-time calls	Scan module/class bodies, decorators, and function defaults; allow standard Triton configuration and decorators (<code>import_time_call</code>)	Prevent side effects before verification
Unsafe calls	Reject <code>open</code> , dynamic code, process, and subprocess calls	Limit common execution escape paths
Reference calls	Calls such as <code>reference</code> , <code>ref_forward</code> , or <code>torch.forward</code> ; reason <code>calls_suspicious_function</code>	Block routing through reference code
Torch fallback	Calls, tensor arithmetic, data views, and non-metadata tensor methods in reachable wrappers; reason <code>obvious_torch_fallback</code>	Reject PyTorch compute hidden behind aliases or helpers
Triton launch	Match a JIT kernel to a reachable indexed launch	Reject import-only or unreachable Triton usage

Timing sessions and cache-state perturbation. A measurement session is a separate loop in the same process and GPU run, not a process-independent replicate. Inputs are regenerated with seed `1234 + session_index`, and a fresh timer/cache-flusher object is used. The cache flusher allocates a CUDA float32 buffer on first use and fills it before each measured sample. In the paper configs the buffer is 128 MB and mode `write`. This perturbs cache state; it does not guarantee complete eviction of every cache level. Timing-level intervals describe within-session samples; session-speedup summaries represent the three repeated loops and remain descriptive at this small session count.

2.1 Repeatability Label Definition

Let `s_single` be the original single-run or run-summary speedup vs eager. Let `s_rep` be the repeat median speedup. Let `session_speedups` be per-session speedups from the rigorous benchmark, and let `tau = 0.98` be the default contradictory-session threshold. The implemented labeler does not use CI/IQR thresholds; those statistics are reported where preserved.

Imported fused8 summaries use the legacy repeatability-report labels because those runs were produced before the current rigorous session-label inputs were preserved in the local package. Historical KernelBench files contain fields produced by the rigorous session labeler, but those labels remain invalid as evidence because the surrounding task contract and reference lifecycle were defective. We report both definitions to preserve provenance rather than silently relabeling old artifacts. The rigorous labeler is preferred for corrected runs because it records session-level parity directly in the benchmark summary.

```

legacy repeatability report:
  if s_rep is missing: INSUFFICIENT_DATA
  elif s_rep >= 1.0 and coefficient_of_variation <= 0.10:
    REPEAT_STABLE_WIN
  elif s_single >= 1.0 and s_rep < 1.0:
    SINGLE_RUN_ONLY_WIN
  elif s_rep >= 1.0:
    UNSTABLE
  else:
    BELOW_EAGER

rigorous benchmark summary:
  stable_above_eager =
    median(session_speedups) >= 1.0
    and all(speedup >= tau for speedup in session_speedups)

  if speedup_vs_eager is missing: INSUFFICIENT_DATA
  elif speedup_vs_eager >= 1.0 and stable_above_eager:
    REPEAT_STABLE_WIN
  elif speedup_vs_eager >= 1.0:
    UNSTABLE
  else:
    BELOW_EAGER

```

Thus `bias_relu` is `SINGLE_RUN_ONLY_WIN`: it reached $1.029\times$ in the original fused8 run and fell to $0.976\times$ repeat median. The rigorous path reserves this label for no result because it has no separate single-run observation; an above-eager session median that fails `stable_above_eager` is `UNSTABLE`. Missing CI/IQR fields are not used to infer labels.

The thresholds are implementation defaults, not tuned values. The legacy `CV <= 0.10` rule is inherited from older repeatability reports. The rigorous `tau = 0.98` default allows small session-level timing variation while still requiring an above-eager median. The preserved artifacts do not include the full per-session speedup vectors needed to recompute headline labels under alternative `tau` values, so the paper does not claim threshold robustness.

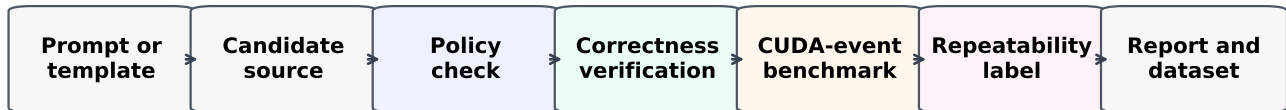
Table 2: Measurement protocol. The KernelBench row describes the affected historical run and is retained only for evaluator auditing.

Study	Timing	Samples $\times$ sessions	Cache	Baseline	Source
fused8	CUDA events	120 $\times$ 3	128 MB	compile	template; Gemini; OpenAI mini
Historical KB	CUDA events	100/120 $\times$ 3 same-process loops	128 MB	compile	affected artifacts

Figure 1 summarizes the evaluation path from candidate construction through artifact export. Each transition is recorded so that a reported speedup can be traced back to its prompt or template, policy outcome, correctness result, and timing samples.

3 Internal Fused8 Study

The fused8 study is the controlled comparison. Deterministic templates, Gemini, and OpenAI mini use the same task suite, verification harness, timing protocol, compiler baseline, and repeatability labels. The suite uses one primary shape regime centered on [4096, 1024], so this section characterizes a controlled regime rather than scaling behavior. Table 3 summarizes the primary results.



Each candidate keeps prompt, response, source, verification, timing samples, and final label.

Figure 1: OpenKernelForge evaluation flow. Candidate artifacts pass through policy checks, correctness verification, CUDA-event benchmarking, repeatability labeling, and report or dataset export.

Table 3: Rigorous fused8 summary. Models beat the compiler baseline at median, but all sources remain below eager at median; deterministic templates remain the strongest floor.

Source	Cand.	Verified	Eager	Compile	Stable wins
template	160	160/160	0.945x	1.079x	residual, bias_gelu, rmsnorm
Gemini	24	23/24	0.923x	1.863x	bias_gelu, rmsnorm
OpenAI mini	24	12/24	0.888x	1.835x	residual

The imported fused8 summaries preserve repeat medians and labels, but not full interval statistics. Historical KernelBench files preserve across-session speedup IQR fields, but the adapter audit makes those intervals non-evidentiary. The uncertainty inventory is summarized in Appendix Table 8.

The template sweep is the strongest overall floor: it verifies all 160 candidates and supplies three repeat-stable wins. Gemini has the highest fused8 model correctness, verifying 23/24 candidates and finding stable wins on `bias_gelu` and `rmsnorm`. OpenAI mini verifies 12/24 candidates, but contributes the only model-over-template stable fused8 win: `residual` at 1.074 $\times$ versus the template repeat median of 1.023 $\times$. Figures 2 and 3 separate source-level coverage from the task-level winners. This comparison should be read as a protocol result under one shared prompt budget, not as a definitive ranking of model families.

Verification rates are reported with descriptive Wilson intervals in Appendix Table 11. Candidates are clustered within eight tasks and candidate budgets differ by source, so candidate-level intervals and counts must not be interpreted as independent-sample model-family rankings. Gemini verifies 23/24 fused8 candidates while OpenAI mini verifies 12/24 under the shared, untuned prompt protocol.

3.1 Compiler Wins and Eager Wins Are Different Claims

Gemini and OpenAI mini are strong relative to `torch.compile max-autotune` on fused8, with median speedups of 1.863 $\times$ and 1.835 $\times$. Both remain below PyTorch eager at median. This is not contradictory. PyTorch eager often dispatches to specialized library kernels or tuned ATen paths, while compiler-generated code can be weaker for the small shapes used here. A generated Triton kernel that beats `torch.compile max-autotune` is not necessarily a universal replacement for eager. The result is a characterization of where generated kernels may help, not a state of the art claim.

3.2 A Single-Run Win That Disappears

The clearest repeatability example is deterministic rather than stochastic. As Figure 4 shows, the `bias_relu` template reaches 1.029 $\times$ in one run but falls to a repeat median of 0.976 $\times$, changing its label to `SINGLE_RUN_ONLY_WIN`. Because the candidate is deterministic, the flip is a measurement and repeatability issue, not model randomness. This motivates assigning repeatability labels even to template-generated kernels.

The imported task-best template table contains four task-best rows above eager in the single-run summary; one of those four falls below eager after repeat measurement. The full 160-candidate per-candidate flip frequency is not preserved in the local artifact package, so the paper treats `bias_relu` as an illustrative case rather than a global flip-rate estimate.

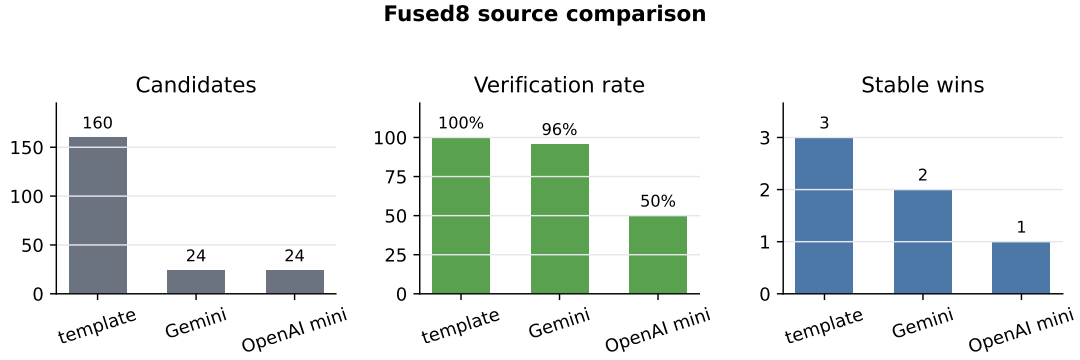


Figure 2: Fused8 source comparison. Templates provide the largest candidate pool and strongest floor; Gemini verifies nearly all candidates; OpenAI mini verifies fewer candidates but contributes one stable win.

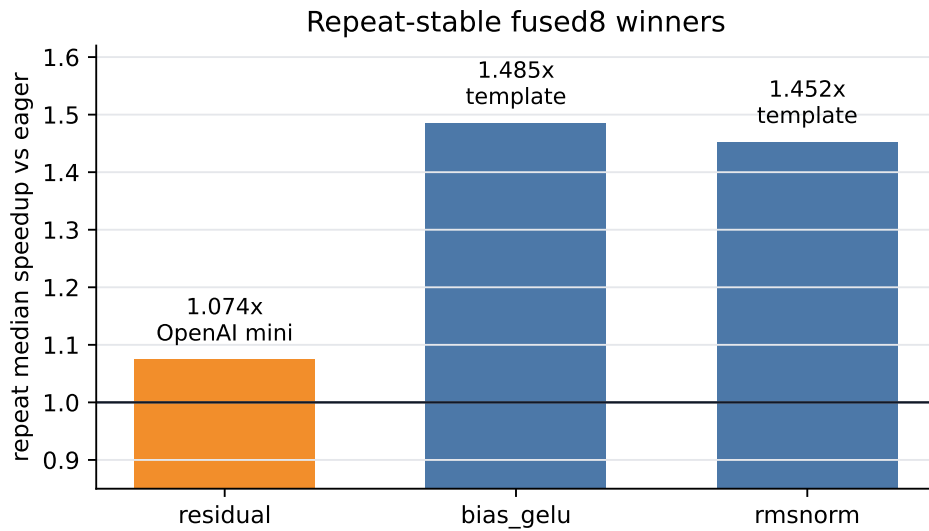


Figure 3: Repeat-stable fused8 winners. Only above-eager repeat-stable wins are shown; the horizontal line marks eager parity.

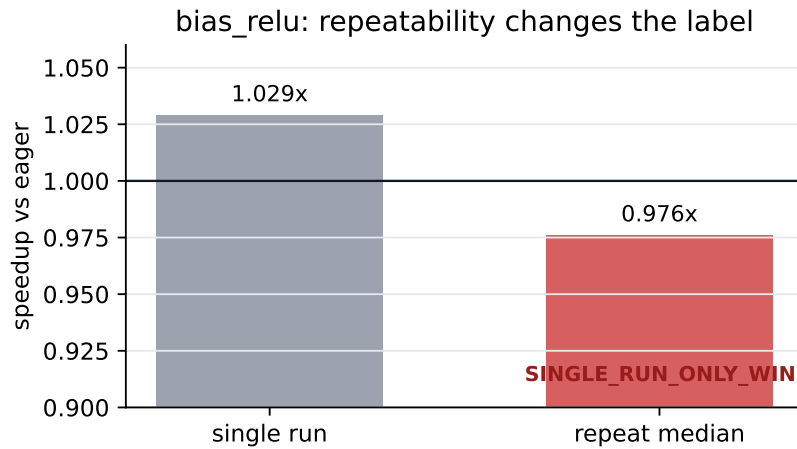


Figure 4: The deterministic `bias_relu` template crosses eager in a single run but falls below eager by repeat median. This is the smallest clean example of why repeatability labels are necessary.

4 KernelBench Adapter Audit

The preserved KernelBench run loaded a 100-task pool, selected 20 tasks after an input-footprint filter, generated one Gemini candidate per task, and attempted one repair on eight failures. Those files provide a useful trace of generation, verification, timing, and failure classification. They do not, however, support an external performance claim under the corrected implementation.

Table 4: Post-hoc KernelBench adapter audit. The corrected implementation removes three defects before external revalidation.

Audit item	Historical behavior	Corrected behavior
State contract	Free <code>forward(*args)</code> received only runtime inputs	Require <code>ModelNew</code> for every official <code>Model</code> task
Reference lifecycle	Rebuilt and transferred <code>Model</code> inside each call	Construct persistent modules from one seeded init-input snapshot
Static policy	Accepted high-level Torch fallback and import-time construction	Apply reachable-call AST policy and require an actual reachable Triton launch

The audit identified two coupled defects. First, the historical prompt required a free `forward(*args)` receiving only `get_inputs()`, although official KernelBench tasks require `ModelNew` initialized from `get_init_inputs()`. The contract mismatch affected every selected task; for parameterized convolution tasks it also left candidates with no valid route to the reference weights. Second, the reference wrapper constructed `Model` and moved it to the device inside every call, while generated candidates were persistent. CUDA-event baseline measurements could consequently include initialization, transfer, or host-submission delay not paid by the candidate.

The corrected adapter materializes one seeded initialization-argument snapshot and constructs persistent reference `Model` and candidate `ModelNew` instances from it before warmup and timing. All official `Model` tasks reject free-function candidates. The benchmark now rotates eager, candidate, and compiled order between sessions, reports session-level uncertainty, and materializes lazy compilation before runtime samples. CPU-only regression tests cover initialization equality, persistent construction, empty-state contract rejection, and failed-import cleanup.

For auditability, the appendix preserves the historical observations: 3/20 candidates verified in the original one-shot run, one of eight repairs verified, and the observed wins concentrated in loss tasks. The combined historical proportion was 4/20 with Wilson interval [0.0807, 0.4160]. These values describe what the affected evaluator recorded; they are not estimates of Gemini accuracy or KernelBench performance. Likewise, historical profiler and clock-recorded diagnostics inherit the old reference lifecycle and are not used for mechanism or speedup claims.

This failure is itself relevant to generated-kernel evaluation. Artifact preservation allowed the candidate contract, reference lifecycle, prompts, and timing records to be inspected together rather than silently relabeling outputs. Corrected GPU revalidation is required before the KernelBench rows can return to the supported-results section.

Appendix Table 12 and the historical funnel and taxonomy figures retain the affected pilot for diagnosis, clearly separated from the controlled fused8 evidence.

5 Discussion

Evaluation role. OpenKernelForge contributes an evaluation protocol and artifact discipline rather than a new kernel-generation model. Its purpose is to make generated-kernel claims auditable: every candidate should be tied to a prompt or template, a policy decision, correctness evidence, benchmark summaries, and a repeatability label. The results suggest that this discipline is useful because the same candidate can pass correctness, beat `torch.compile max-autotune`, lose to eager, or fail repeat verification depending on which claim is being made.

Controlled evidence and evaluator audit. The fused8 study supplies the controlled performance evidence: it compares deterministic templates, Gemini, and OpenAI mini under one task suite and measurement protocol. The KernelBench artifacts now serve a different role. They show that artifact preservation made it possible to identify a state-contract defect and timed model-construction confound after the run. The corrected adapter is implemented and tested, but its GPU results are not yet part of this paper.

What is and is not claimed.

Table 5: Claim boundaries. The paper argues for a measurement and artifact discipline, not for model superiority or leaderboard performance.

Claim supported by this paper	Claim not made
Repeatability-aware evaluation changes which wins are stable.	No state of the art generated-kernel performance claim.
Templates are a useful performance floor in the fused8 study.	No benchmark-wide KernelBench result or estimate.
Historical KernelBench artifacts expose adapter and policy failure modes.	No validated KernelBench accuracy or speedup estimate from the affected runs.
Corrected task contracts are required before external results are promoted.	No proof that LLM speedups are broadly repeatability-fragile.
Compiler-baseline and eager-baseline speedups are distinct claims.	No deployment-cost or compile-amortization analysis.

Stable wins as characterization, not rankings. Stable-win counts are descriptive. Counts such as three template wins, two Gemini wins, or one OpenAI mini win identify candidates that survive the protocol under a fixed budget; they are too small to support statistically significant model rankings. The stronger finding is structural: generated-kernel reports should distinguish correctness, compiler-baseline speedup, eager speedup, and repeat-stable speedup.

Implications for generated-kernel evaluation. The protocol should be treated as a useful default for generated-kernel evaluation, not as a universal prescription for every deployment setting. In exploratory settings, single-run speed can remain a useful search signal. For paper-facing or dataset-facing claims, however, the evidence here supports preserving artifacts and assigning conservative repeatability labels before promoting a candidate as fast.

6 Related Work

Generated GPU kernels and kernel agents. Kernel-generation systems translate tensor programs into CUDA or Triton implementations and must satisfy both correctness and performance constraints. KernelBench frames this as a model-evaluation problem at benchmark scale [9]. CUDA-L1 studies feedback-driven CUDA optimization as a learning problem [5]. OpenKernelForge is narrower: it is not a new model, training method, or leaderboard submission. It provides evaluation infrastructure for deciding which generated-kernel speedups are supported by preserved artifacts and repeatability-aware measurement.

Kernel generation benchmarks. KernelBench asks whether LLMs can write efficient GPU kernels across many tasks [9]. KernelBench-Verified subsequently shows that baseline configuration and narrow correctness distributions can overstate generated-kernel performance [13]. Its hidden-test and TF32 analyses complement the present focus on task-state contracts, lifecycle isolation, repeatability, and artifact provenance. This paper does not report a validated KernelBench score. Instead, it uses preserved artifacts from a capped historical pilot to audit policy checks and evaluator behavior. That audit motivated the corrected **ModelNew** adapter and shows why evaluator validity must be established before model capability is inferred.

Triton and GPU DSLs. Triton exposes GPU programming concepts through a Python-embedded DSL [11]. Other systems such as Halide [10] and TVM [2] show the value of scheduling languages and compiler infrastructure for high-performance kernels. FlashAttention illustrates how memory movement and IO-aware design can dominate GPU-kernel performance [4]. OpenKernelForge uses Triton as the candidate target because generated code can remain close to Python while expressing GPU tiling and memory behavior.

Compiler baselines and PyTorch compilation. PyTorch 2 and TorchInductor make compiler-generated kernels part of the practical baseline for tensor programs [1]. This paper reports both PyTorch eager and `torch.compile max-autotune` because they answer different questions. Eager may call specialized library paths, while compile may expose weaker compiler-generated paths for some shapes.

Measurement reliability. Systems researchers have long shown that performance measurements can change under seemingly irrelevant experimental details. Mytkowicz et al. demonstrate that measurement bias can produce wrong conclusions without any obvious methodological mistake [7]. Stabilizer argues for statistically sound performance evaluation under modern architectural nondeterminism [3]. Touati similarly emphasizes statistical methodology for speedup claims in program optimization [12]. GPU timing also requires explicit synchronization and careful separation of warmup, compilation, and measured runtime. CUDA events provide device-side elapsed-time measurement [8]. The novelty here is not CUDA timing itself; it is integrating repeatability labels, artifact preservation, static policy checks, and generated-kernel verification into one evaluation layer.

Repair and feedback for generated code. Verifier feedback is a natural source of repair context. Self-Refine studies iterative refinement through feedback for generated outputs [6]. The preserved historical repair pass is narrower: it provides verifier errors and the previous candidate code to Gemini once. We retain it to document the evaluator workflow, not as corrected evidence of repair effectiveness.

Positioning. Appendix Table 19 summarizes the relation between OpenKernelForge and adjacent areas. KernelBench and CUDA-L1 focus on generated-kernel capability and improvement; Triton, Halide, TVM, and TorchInductor provide compiler and DSL context; systems-measurement work motivates statistical caution. OpenKernelForge combines these threads as a repeatability-aware evaluation and artifact-preservation layer, including auditability of the evaluator itself.

7 Limitations

The fused8 benchmark is internal and small. It is useful as a controlled study, but it is not a substitute for broad external benchmarking. The fused8 suite uses one primary shape regime, centered on [4096, 1024] tensors. Its results characterize this controlled regime and should not be interpreted as evidence about scaling across batch sizes, feature dimensions, or sequence lengths. The historical KernelBench pilot used a capped, non-random subset selected with an incomplete memory lower bound. More importantly, its free-function candidate contract violated the official ModelNew lifecycle, could not represent parameterized task state, and paired candidates with reference modules reconstructed inside timed calls. We therefore make no KernelBench correctness or speedup claim from those runs. Corrected-adapter GPU revalidation is outstanding.

The supported model budgets are small: 24 fused8 candidates for Gemini and 24 for OpenAI mini. The historical KernelBench artifacts contain one Gemini candidate per selected task and one candidate per selected repair, but the affected adapter prevents using those budgets for a model comparison. The OpenAI mini prompt was not separately tuned, so its lower fused8 verification rate should be interpreted as a result under the shared protocol rather than a definitive model-family comparison.

Candidate search creates multiplicity. The template sweep evaluates many variants, and model runs select from generated candidates. Repeatability labels reduce the risk of promoting noisy single-run artifacts, but they do not eliminate selection bias or multiple-comparisons effects. The fused8 effects are reported only for their controlled setting; no family or model superiority claim is made.

Some imported fused8 summaries preserve repeat medians and labels but not complete p25/p75 or bootstrap intervals; those rows are marked as not preserved. Three same-process sessions provide a practical stability check rather than a high-powered variance estimate. Within-session sample counts are high, but between-session uncertainty remains limited by the session count. Current code reports session-level summaries and rotates measurement order; historical runs used pooled timing intervals and fixed order.

Repeatability thresholds are fixed implementation defaults rather than tuned constants. The legacy labeler uses $CV \leq 0.10$; the rigorous session labeler uses $\tau = 0.98$. The local artifacts do not preserve the per-session vectors needed for a full threshold-sensitivity analysis, so threshold robustness is not claimed.

The study uses one primary benchmark GPU class. Historical loss-family profiler and clock-recorded diagnostics inherit the affected KernelBench reference lifecycle and are retained only as debugging artifacts. GPU and memory clocks were not locked for the original campaign; dynamic boost, power state, and thermal behavior may contribute to run-to-run variation. CUDA graphs were not used. For small-shape workloads, graph capture can reduce launch overhead and may change relative performance, so comparisons should be interpreted under ordinary eager and compiled execution rather than graph-captured deployment.

Current compile accounting measures wrapper creation plus the first synchronized materializing call, while steady-state runtime excludes that call. Historical compile-time fields measured wrapper construction alone or were null and are not interpreted. Compile-cost amortization remains outside the paper.

Static policy checks are not a security sandbox. Candidate modules execute locally after AST checks, so untrusted public submissions require disposable worker or container isolation and an external timeout.

These constraints limit the scope to methodology and bounded characterization rather than leaderboard comparison.

8 Conclusion

OpenKernelForge demonstrates a repeatability-aware evaluation layer for LLM-generated Triton kernels. The internal fused8 study shows that deterministic templates remain a strong floor, that model kernels can beat `torch.compile max-autotune` while losing to eager at median, and that a deterministic single-run win can disappear under repeat measurement. The KernelBench audit shows a complementary requirement: task-state contracts and baseline lifecycles must be validated before external benchmark rows are treated as evidence.

The central lesson is methodological. Generated-kernel reports should distinguish correctness, compiler speedup, eager speedup, and repeat-stable speedup, and they should preserve enough state to audit those decisions. The corrected harness provides a useful default for future evaluation. Corrected external validation remains necessary before making KernelBench claims.

References

- [1] Jason Ansel et al. PyTorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 929–947. ACM, 2024.
- [2] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 2018.
- [3] Charlie Curtsinger and Emery D. Berger. STABILIZER: Statistically sound performance evaluation. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’13, pages 219–228. ACM, 2013.
- [4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness, 2022. arXiv:2205.14135.
- [5] Xiaoya Li, Albert Wang, Guoyin Wang, Jiwei Li, and Chris Shum. CUDA-L1: Improving CUDA optimization via contrastive reinforcement learning, 2025. arXiv:2507.14111.
- [6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.
- [7] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’09, pages 265–276. ACM, 2009.
- [8] NVIDIA. *CUDA Runtime API: Event Management*. NVIDIA, 2026. CUDA documentation for event creation, synchronization, and elapsed-time measurement; https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__EVENT.html.
- [9] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels?, 2025. arXiv:2502.10517.
- [10] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2013.

- [11] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. ACM, 2019.
- [12] Sid Touati. Towards a statistical methodology to evaluate program speedups and their optimisation techniques, 2009. arXiv:0902.1035.
- [13] Yunxiang Zhang, Ping Yu, Jianyu Wang, Max (Xiangjun) Fan, Julian Reed, Azalia Mirhoseini, and Will Su. KernelBench-Verified: Do LLM-generated kernels actually beat PyTorch?, 2026. arXiv:2607.16241.

A Supplementary Material

This appendix keeps task-level details out of the main body. Raw prompts, responses, candidate files, verification traces, and benchmark samples remain in the run artifacts.

A.1 Example generated candidates

The examples below are abridged excerpts from the historical KernelBench run. They make the adapter and verifier audit concrete; they are not corrected-adapter results.

Historically accepted one-shot candidate: CrossEntropyLoss. The historical one-shot Gemini candidate for CE passed the original policy and correctness checks and was labeled `REPEAT_STABLE_WIN`. Its recorded speedups are retained in the raw artifact but are provisional because the reference lifecycle was defective. Static inspection shows a row-wise log-sum-exp pattern followed by target-logit gather and a Torch mean reduction. The current `ast-v5` policy rejects that high-level reduction when fallback is disabled. The preserved source is under `artifacts/runpod_imports/runs/20260520_202314/candidates/.../candidate_000.py`.

```
@triton.jit
def _cross_entropy_kernel(logits_ptr, targets_ptr, loss_ptr,
                          n_rows, n_cols, BLOCK_SIZE: tl.constexpr):
    row_idx = tl.program_id(0)
    ...
    log_sum_exp = max_val + tl.log(sum_exp)
    target = tl.load(targets_ptr + row_idx)
    target_logit = tl.load(logits_ptr + row_idx * n_cols + target)
    tl.store(loss_ptr + row_idx, log_sum_exp - target_logit)

def forward(predictions, targets):
    n_rows, n_cols = predictions.shape
    loss = torch.empty(n_rows, device=predictions.device,
                      dtype=predictions.dtype)
    _cross_entropy_kernel[(n_rows,)](...)
    return loss.mean()
```

Failed one-shot candidate: 4D tensor matmul. The historical `matmul_4d` candidate passed static policy but failed verification before timing with a Triton compilation error. It attempts a blocked matmul over a flattened 4D tensor, but pointer and mask ranks do not match. The verifier records `triton.compiler.errors.CompilationError` with summary `Cannot broadcast, rank mismatch: ['1','1','32'], ['16','1','1','32']`. The failure was labeled high repairability because it is localized to pointer arithmetic and mask shape; the single repair attempt did not verify.

```
A_ptr_base = A_ptr + (rn[:, None, None, None] // (256 * 512)) * stride_ab + ...
acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
for k in range(0, K, BLOCK_K):
    a = tl.load(
        A_ptr_base + (rk[None, None, :] + k) * stride_al,
        mask=rk[None, None, :] + k < K,
        other=0.0,
    )
    b = tl.load(B_ptr + (rk[:, None] + k) * stride_bk + rn[None, :], ...)
    acc += tl.dot(a.reshape(BLOCK_M, BLOCK_K), b)
```

Table 6: Verifier-error example. The failure taxonomy records both the concrete exception and the repairability decision.

Task	Category	Repairability	Verifier summary
matmul_4d	Triton compile	high	tl.load mask and pointer ranks do not broadcast; selected for repair because the failure is localized to pointer arithmetic and constexpr mask shape.

A.2 Historical KernelBench diagnostic figures

Figures 5 and 6 summarize the affected run’s output funnel and recorded failure taxonomy. They are retained to document the evaluator audit, not to support corrected-adapter performance claims.

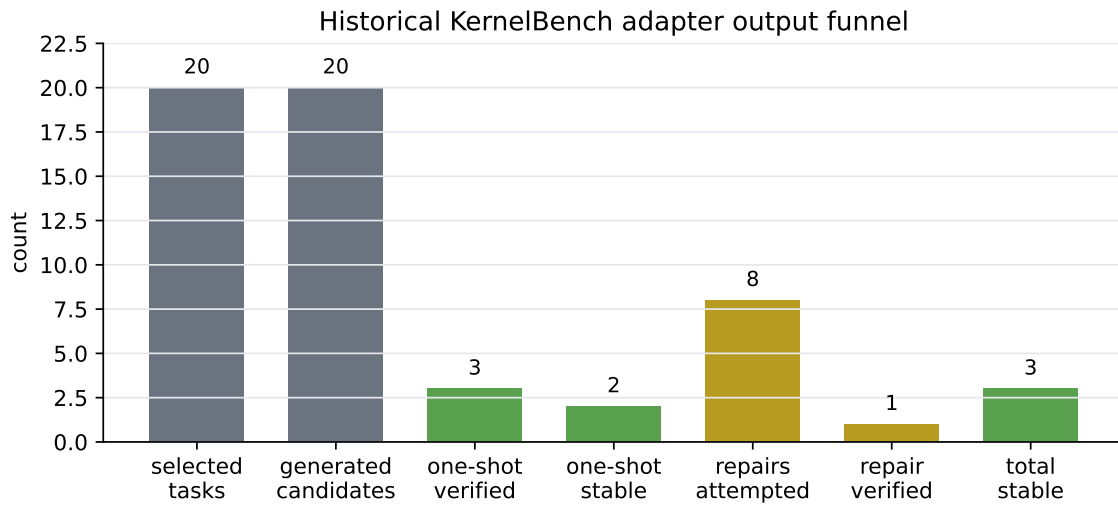


Figure 5: Output funnel from the affected historical adapter. Counts describe preserved artifacts, not corrected-adapter model accuracy.

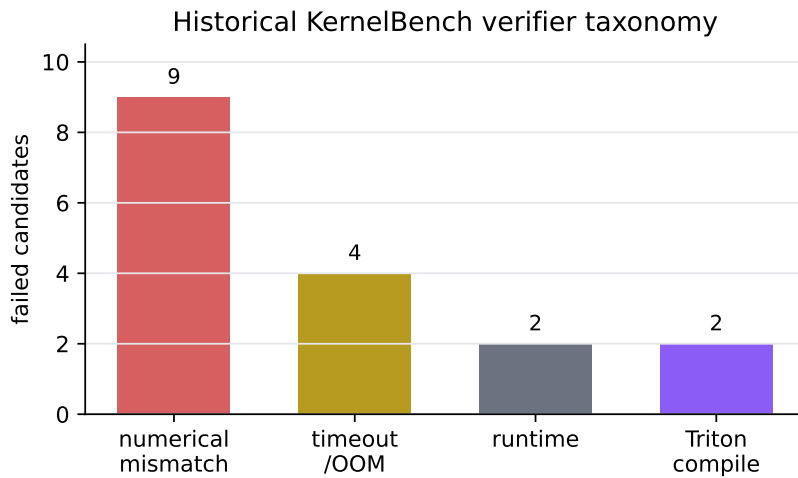


Figure 6: Failure categories recorded by the historical verifier. Contract-invalid parameterized candidates make this a diagnostic taxonomy rather than a model-capability estimate.

A.3 Fused8 task and uncertainty details

Table 7: Fused8 task shapes and tolerances. All tasks use float32 and deterministic seeded input generation.

Task	Reference operation	Inputs	rtol	atol
bias_relu	<code>relu(x + bias)</code>	x [4096,1024], bias [1024]	1e-5	1e-6
sigmoid	<code>sigmoid(x) * z</code>	x [4096,1024], z [4096,1024]	1e-5	2e-6
add_relu	<code>relu(x + y)</code>	x [4096,1024], y [4096,1024]	1e-5	1e-6
residual	<code>relu(x + residual + bias)</code>	x [4096,1024], residual [4096,1024], bias [1024]	1e-5	1e-6
bias_gelu	sigmoid-approximate GELU after bias add	x [4096,1024], bias [1024]	2e-4	2e-5
row_sum	sum over final dimension	x [4096,1024]	1e-4	1e-4
layernorm	layer normalization over final dimension	x [4096,1024], weight [1024], bias [1024]	2e-4	2e-4
rmsnorm	RMS normalization over final dimension	x [4096,1024], weight [1024]	2e-4	2e-4

Table 8: Fused8 task-level repeatability summary. Imported fused8 summaries preserve medians and labels, but not all interval statistics.

Task	Winner	Source	Repeat median	Uncertainty	Interpretation
bias_relu	none	n/a	0.976x	not preserved	single-run template win fell below eager
sigmoid	none	n/a	0.997x	std 0.029, CV 0.030	nearest model remained below eager
add_relu	none	n/a	0.968x	std 0.003, CV 0.003	nearest model remained below eager
residual	OpenAI mini	llm	1.074x	std 0.048, CV 0.045	OpenAI mini is the only model-over-template stable win
bias_gelu	template	template	1.485x	not preserved	template remains stronger than Gemini
row_sum	none	n/a	0.674x	not preserved	all verified candidates below eager
layernorm	none	n/a	0.791x	not preserved	all verified candidates below eager
rmsnorm	template	template	1.452x	not preserved	template remains strongest by repeat median

Table 9: Fused8 uncertainty recovery status. The local package preserves medians and labels, but the full rigorous fused8 run directories are still required for interval and all-candidate flip recovery.

Source	Run	IQR	Bootstrap CI	Flip pairs	Notes
template	20260520_155839	not preserved	not preserved	not preserved	full run directory is missing locally and under artifacts/runpod_imports
Gemini	20260520_163344	not preserved	not preserved	not preserved	full run directory is missing locally and under artifacts/runpod_imports
OpenAI mini	20260520_163607	not preserved	not preserved	not preserved	full run directory is missing locally and under artifacts/runpod_imports

Table 10: Qualitative fused8 eager-baseline notes. These notes are based on reference-source inspection rather than profiler traces.

Task/family	Likely eager path	Conf.	Caveat
elementwise fused8	ATen elementwise kernels and broadcasting paths	medium	qualitative source inspection only; no profiler traces in local package
row_sum	ATen reduction kernel	medium	exact kernel path not recorded
layernorm/rmsnorm	ATen normalization and reduction operations	medium	no Nsight or torch.profiler attribution is preserved

Table 11: Statistical appendix summary. Fused8 intervals describe supported fixed-budget observations; KernelBench intervals describe affected historical evaluator output only.

Study	Source	Verified	Rate	Wilson 95% CI
fused8	template	160/160	1.0000	[0.9766, 1.0000]
fused8	Gemini	23/24	0.9583	[0.7976, 0.9926]
fused8	OpenAI mini	12/24	0.5000	[0.3143, 0.6857]
Historical KB (affected): pilot	Gemini one-shot	3/20	0.1500	[0.0524, 0.3604]
Historical KB (affected): repair1	Gemini repair	1/8	0.1250	[0.0224, 0.4709]
Historical KB (affected): combined	Gemini one-shot + repair1	4/20	0.2000	[0.0807, 0.4160]
Task-best template flip summary: 1 of 4 single-run-above-eager task-best rows fall below eager after repeat measurement. Full 160-candidate flip frequency is not preserved.				

A.4 Historical KernelBench audit details

Table 12: Family-level output from the affected historical KernelBench evaluator. The apparent loss-family concentration is diagnostic and not a corrected-adaptor accuracy estimate.

Family	Sel.	1-shot ok	1-shot stable	Repair tried	Repair ok	Combined ok	Combined stable
convolution	7	0	0	0	0	0	0
matmul	7	1	0	6	0	1	0
pooling	3	0	0	1	0	0	0
loss	3	2	2	1	1	3	3
Memory-cap skips encountered before the 20-task cap: activation 7; kernelbench.ll 6; loss 3; normalization 6; pooling 3; reduction 6. The selector takes the first feasible tasks in deterministic family-round-robin order; it is not random and does not scan the remaining pool after reaching the cap.							

Table 13: Historical KernelBench memory filtering. The estimate was a lower bound and the affected run is retained only to document selection provenance.

Category	Family	Count	Shape examples
selected feasible	all	20	see selected task appendix
skipped memory cap	all	31	[4096,393216]; [4096,393216]
selected feasible	activation	0	not preserved
skipped memory cap	activation	7	[4096,393216]; [4096,393216]
selected feasible	convolution	7	[256,3,224,224]; [16,3,64,64,64]
skipped memory cap	convolution	0	not preserved
selected feasible	kernelbench.l1	0	not preserved
skipped memory cap	kernelbench.l1	6	[4096,393216]; [4096,393216]
selected feasible	loss	3	[32768,4096]; [16384,16384]
skipped memory cap	loss	3	[32768,32768]; [32768,32768]
selected feasible	matmul	7	[16,1024,2048]; [8,256,512,256]
skipped memory cap	matmul	0	not preserved
selected feasible	normalization	0	not preserved
skipped memory cap	normalization	6	[64,64,512,512]; [112,64,512,512]
selected feasible	pooling	3	[64,192,65536]; [32,64,512,512]
skipped memory cap	pooling	3	[16,32,128,128,128]; [16,64,2048,2048]
selected feasible	reduction	0	not preserved
skipped memory cap	reduction	6	[128,4096,4095]; [128,4096,4095]

Table 14: Repairability categories used by the affected historical evaluator. They document selection provenance but do not validate the resulting repair rows.

Label	Operational criterion	Selected	Observed	Notes
high	wrong signature/input count, shape mismatch, Triton compile error, or numerical mismatch in matmul/loss tasks	8	8	historical selection provenance; not repair-effectiveness evidence
medium	runtime exception or other category not explicitly high/low	0	0	historical analyzer category; absent from the initial taxonomy summary
low	convolution numerical mismatch, timeout/OOM, verification harness issue, unsupported pattern, or invalid forward state	0	9	historical selection provenance only

Table 15: Compile-time field availability. Historical KernelBench compile fields use obsolete accounting or are missing and are not interpreted.

Stage	Verified	Compile times	Compile summary	Runtime summary
one-shot Gemini	3	0	not preserved	median 0.3788
repair1	1	0	not preserved	median 4.1367

Table 16: Task aliases from the affected historical KernelBench adapter run. The subset and its outputs are retained for auditability, not external performance inference.

Task alias	Family	Task alias	Family
conv2d_square	conv	matmul_3d	matmul
maxpool1d	pool	conv3d_square	conv
matmul_4d	matmul	maxpool2d	pool
conv2d_asym_input	conv	CE	loss
matmul_diagonal	matmul	conv2d_asym_kernel	conv
matmul_symmetric	matmul	avgpool1d	pool
conv_transpose2d	conv	kldiv	loss
matmul_upper_tri	matmul	conv_transpose3d	conv
triplet	loss	matmul_lower_tri	matmul
conv3d_asym_input	conv	matmul_transposed_A	matmul

Table 17: Outcomes recorded by the affected historical KernelBench evaluator. Current policy and state-contract corrections prevent interpreting these rows as model capability.

Task/category	Family	Label	Result	Note
CE	loss	stable	1.992x	IQR 0.0022
triplet	loss	stable	4.176x	IQR 0.0005
matmul_diagonal	matmul	below	0.984x	IQR 0.0016
numeric failures	mixed	failed	9 tasks	dominant failure category
timeout/OOM	mixed	failed	4 tasks	candidate execution failures
runtime	conv	failed	2 tasks	implementation exceptions
Triton compile	mixed	failed	2 tasks	compile errors

Table 18: Repair outcomes recorded by the affected historical evaluator. The rows characterize its failure workflow and are not corrected-adaptor results.

Task	Original failure	Repairability	Repair result
matmul.3d	numeric	high	failed
matmul_symmetric	numeric	high	failed
matmul_upper_tri	numeric	high	failed
matmul_lower_tri	numeric	high	failed
matmul_transposed_A	numeric	high	failed
kldiv	numeric	high	stable, 1.843x
matmul.4d	Triton compile	high	failed
avgpool1d	Triton compile	high	failed

A.5 Positioning summary

Table 19: Compact positioning. OpenKernelForge contributes an evaluation protocol and artifact discipline rather than a new generator or leaderboard result.

Area	Main role	Relation to OpenKernelForge
KernelBench	benchmark	scale-oriented model evaluation
CUDA-L1	model/system	feedback-driven CUDA optimization
Triton / GPU DSLs	DSL/compiler	candidate implementation target
PyTorch 2 / TorchInductor	compiler baseline	practical compile baseline
Systems measurement methodology	measurement	repeatability and bias control
Verifier-guided repair / Self-Refine-style feedback	repair	one capped verifier-feedback pass
OpenKernelForge	this paper	evaluation protocol and artifact discipline

B Prompt and Decoding Appendix

The paper-facing model runs use fixed prompt templates and small candidate budgets. Full prompts, raw responses, extracted candidates, and backend metadata are saved in run artifacts when those artifacts are available.

Fused8 generation. Fused8 model candidates use prompt version `v2.task_skeletons`. The prompt requires a module-level `forward(*args)`, allows only `torch`, `triton`, and `triton.language` as `tl` imports, includes the task description, `dtype`, tolerance, benchmark shape, task hints, and PyTorch reference source, and instructs the model not to fake outputs, cache expected tensors, read hidden files, call the reference function, or use plain PyTorch fallback code as the main implementation. The system prompt is: “You generate concise Python candidate kernels for local verification. Return code that can be imported as a Python module.”

Fused8 settings. Gemini uses model `gemini-3.1-flash-lite`, OpenAI-compatible Gemini endpoint, temperature 0.2, top-p 0.95, max tokens 4096, one attempt, and three candidates per task. OpenAI mini uses model `gpt-5.4-mini`, OpenAI-compatible chat completions, null temperature/top-p, `extra_body.max_completion_tokens = 2048`, one attempt, and three candidates per task. Both disable torch fallback, stop-after-first-correct, repair loops, template guidance, and performance search.

These are configured API strings, not inferred provider revisions. Provider response `model` fields are not preserved in the local fused8 artifacts.

KernelBench generation. KernelBench prompts include the task source, benchmark metadata, tolerance, and an explicit candidate contract. The corrected adaptor requires `ModelNew(torch.nn.Module)` for every official task that defines `Model`; `ModelNew` receives the same seeded `get_init_inputs()` snapshot as the reference, and its `forward` receives `get_inputs()`. Module-level `forward(*args)` candidates are limited to local synthetic tasks with an explicit `reference_fn` interface. The prompt forbids reference/task calls and permits Torch only for allocation, metadata inspection, parameter storage, and Triton launch wrappers. The preserved 20260520 prompts used the older free-function-only contract and are retained verbatim for auditability; they are not presented as the corrected protocol. Gemini settings were temperature 0.2, top-p 0.95, max tokens 4096, and one candidate per task.

KernelBench JSONL records preserve the configured model field `gemini-3.1-flash-lite`; they do not preserve a separate provider-returned model-version field.

Repair prompts. The historical repair prompts include the original task source, failed candidate, verification error, traceback summary, failure category, and suggested repair instruction. They prioritize correctness over performance and use one Gemini response per selected task. Because they inherited the obsolete free-function contract, they document prompt provenance rather than a validated repair protocol. Corrected repair prompts must carry the same `ModelNew` state contract as corrected one-shot generation.